Lecture 1

Introduction & Data Representation — Lecture Notes

I. **Languages & Translation.** Machine, assembly and high-level languages; the layered view; compiling, assembling, linking and loading.

II. **Why Assembly?** What assembly is, what it is for, and how it maps to machine code.

III. **Data Representation.** Number systems, conversions, binary addition and integer storage sizes.

IV. **Digital Logic.** Gates, Boolean algebra and operator precedence.

Notation. [S5] means slide 5 of Lecture 1.

Mahmoud AboZied
Teaching Assistant

Academic Year 2026 / 2027

Contents

Part I · Languages & Translation **3**
1 · Introduction [S3, 4] 3
2 · The Layered (Virtual-Machine) View [S5, 8, 14] 3
3 · Compiling, Assembling, Linking, Loading [S9, 10] 4

Part II · Why Assembly? **5**
4 · Why Learn Assembly Language? [S12, 13] 5
5 · Assembly Language [S15, 16] 5
6 · The Assembler [S17] 5
7 · Machine Language [S18, 19] 5
8 · Machine vs. Assembly vs. High-Level [S20, 21, 22, 23] 5

Part III · Data Representation **7**
9 · Number Systems [S24] 7
10 · Conversions [S25, 26, 29, 30] 7
11 · Binary Addition [S27] 7
12 · Integer Storage Sizes [S28] 8

Part IV · Digital Logic **9**
13 · Digital Logic & Boolean Algebra [S31, 32, 33] 9
14 · Operator Precedence [S34] 9
Key Points to Remember 9

Part I · Languages & Translation

1 · Introduction [S3, 4]

- The first programs were written in **machine language**, then in **assembly language**.
- Today most programs are written in **high-level languages** (C, C++, Java ...).

The same task at three levels — *"Display the sum of A times B plus C"*:

C++

```
cout << (A * B + C);
```

Assembly

```
mov  eax, A
mul  B
add  eax, C
call WriteInt
```

Intel machine code

```
A1 00000000
F7 25 00000004
03 05 00000008
E8 00500000
```

One C++ statement became **four** assembly instructions, and each assembly instruction became **exactly one** machine instruction.

2 · The Layered (Virtual-Machine) View [S5, 8, 14]

L4	**High-Level Language** *Human-friendly: C, C++, Java, Fortran, Cobol*
L3	**Assembly Language** *Mnemonics: MOV, ADD, MUL ...*
L2	**Instruction Set Architecture (ISA)** *The CPU's native machine language*
L1	**Digital Logic** *Gates & transistors: the CPU hardware*

Programming languages are built in **layers**. A program at one level runs by

- **Interpretation**: the level below executes it one instruction at a time, or
- **Compilation (translation)**: it is converted completely into the level below first.

Every CPU has a **built-in interpreter** for its own **instruction set** (ISA): the binary language it is programmed in.

The key concepts [S8]

1. A **high-level language** (C, C++, Fortran, Cobol) is **compiled** into assembly language.
2. The **assembly language** (for one specific CPU) is **assembled** into binary machine language.
3. The binary machine language is **interpreted** by the CPU.
4. The CPU (Intel, AMD ...) uses **digital logic circuits** to do the interpretation and produce the results.

How real languages use the levels [S6, 7]

Step	C / C++	Java
Write	Programmer writes a C program in an editor	Programmer writes a Java program
Translate	**C compiler** → assembly (object file)	**Java compiler** (`javac`) → **byte code** (`.class`)
Assemble	**MASM** → Intel binary code	—
Run	**x86 CPU** executes (interprets) the binary	**JVM** (an interpreter) runs the byte code

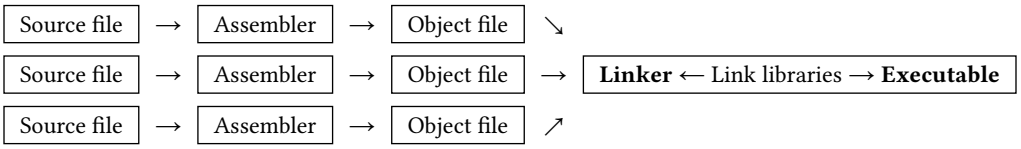
A hierarchy of languages [S14]

Application programs → **high-level languages** → assembly language → machine language → microprogram control → hardware. High-level languages are **machine-independent**. Assembly and machine language are **machine-specific**.

3 · Compiling, Assembling, Linking, Loading [S9, 10]

Tool	Input → Output	Remember
Compiler	High-level language → assembly / machine code	Directly, or indirectly through an assembler. [S20]
Assembler	Assembly → machine (object) code	Translates **each source file separately**. Example: **MASM**.
Linker	Object files + **link libraries** → **executable**	Joins all the parts and resolves names.
Loader	Executable on disk → program in memory	Part of the OS.

Four basic steps. **Assembling** (translate to binary) → **Linking** (join the parts, resolve names) → **Loading** (move the program into memory) → **Execution** (run it). Assembling **alone** does not create a program that can be executed: it must be linked with the library first.



Part II · Why Assembly?

4 · Why Learn Assembly Language? [S12, 13]

- To learn how **high-level code is translated** into machine language.
- To learn the computer's **hardware** by direct access to memory, the video controller, the sound card, the keyboard ...
- To **speed up** applications by direct access to hardware.
- To **communicate with hardware**: drivers and embedded systems.
- To write **faster code**, especially games and graphics.
- Some procedures need **complete access** to the hardware to be efficient, which is usually impossible in a high-level language.
- **Better understanding** of programming: what happens "under the hood".
- Someone needs to develop **future** computer systems and programming languages.

5 · Assembly Language [S15, 16]

- Designed for a **specific family of CPUs** (e.g. Intel x86): a **low-level, processor-specific** language designed to match the processor's machine instruction set.
- An instruction is a **mnemonic** (a simplified command word) followed by the needed data:

```
mov eax, A     ; move into register EAX the contents of the location called A
```

- Each mnemonic (instruction) is equivalent to **a single binary CPU instruction**: one assembly instruction matches **exactly one** machine instruction.

6 · The Assembler [S17]

An **assembler** is a program that translates programs written in assembly language into **machine language** that the computer can execute. Example: **MASM** (Microsoft **Macro** Assembler).

7 · Machine Language [S18, 19]

- The **lowest-level** language, and the **only** one the computer understands **without translation**.
- Easy for computers, but almost impossible for humans: it is **entirely binary digits**.
- Every CPU has its **own** machine language.

Each instruction = an **opcode** (operation code) + **zero or more operands**.

Opcode	Operand	Meaning
`40`	—	Increment the AX register
`05`	`0005`	Add 0005 to AX

8 · Machine vs. Assembly vs. High-Level [S20, 21, 22, 23]

	Machine language	Assembly language
Level	Lowest; executed **directly by hardware**	Slightly higher-level
Form	Binary code: 1s and 0s	Mnemonics: readable instructions
CPU	Processor-specific	Processor-specific
Mapping	—	**One-to-one** with machine instructions

Translating a statement. *D is assigned the sum of A times B plus 10* → HLL: `D = A * B + 10`

```
mov eax, A                                  A1 00404000
mul B                                       F7 25 00404004
add eax, 10                                 83 C0 0A
mov D, eax                                  A3 00404008
```

HLL → machine is not one-to-one: one high-level **statement** is translated into **one or more** machine instructions. Assembly → machine **is** one-to-one. [S23]

Part III · Data Representation

9 · Number Systems [S24]

Computers work with **binary** data, sometimes written in octal (base 8) or hexadecimal (base 16).

System	Base	Digits
Binary	2	0, 1 (1 = true / current flowing, 0 = false / no current)
Octal	8	0 – 7
Decimal	10	0 – 9
Hexadecimal	16	0 – 9, A – F (A = 10 … F = 15)

10 · Conversions [S25, 26, 29, 30]

Binary → decimal (weighted positional notation) *[S25]*

$$\text{dec} = (D_{n-1} \times 2^{n-1}) + (D_{n-2} \times 2^{n-2}) + \ldots + (D_1 \times 2^1) + (D_0 \times 2^0)$$

Example: `00001001b` = $(1 \times 2^3) + (1 \times 2^0) = 8 + 1 = \mathbf{9}$.

Decimal → binary (repeated division by 2) *[S26]*

Division	Quotient	Remainder
37 / 2	18	1
18 / 2	9	0
9 / 2	4	1
4 / 2	2	0
2 / 2	1	0
1 / 2	0	1

Divide until the quotient is 0. Read the remainders from the **bottom up**: 37 = **100101b**.

Binary → hexadecimal *[S29]*

Each hex digit corresponds to **4 bits**. Group the bits in fours from the right:

`0001`	`0110`	`1010`	`0111`	`1001`	`0100`
1	**6**	**A**	**7**	**9**	**4**

`00010110101001111001010 0b` = **16A794h**.

Hexadecimal → decimal *[S30]*

Multiply each digit by its power of 16: $\text{dec} = (D_3 \times 16^3) + (D_2 \times 16^2) + (D_1 \times 16^1) + (D_0 \times 16^0)$.

- `1234h` = 1 × 4096 + 2 × 256 + 3 × 16 + 4 = **4 660**.
- `3BA4h` = 3 × 4096 + 11 × 256 + 10 × 16 + 4 = **15 268**.

11 · Binary Addition [S27]

Start at the **LSB** (bit 0) and add each pair of digits, including the carry if there is one.

```
carry:          1
        0000 0100     (4)
      + 0000 0111     (7)
        ---------
        0000 1011    (11)
bit:    7654 3210
```

12 · Integer Storage Sizes [S28]

Type	Size	Bits
Byte	1 byte	8
Word	2 bytes	16
Doubleword	2 words = 4 bytes	32
Quadword	4 words = 8 bytes	64
Double quadword	8 words = 16 bytes	128

Storage type	Range
Unsigned byte	0 … 255
Signed byte	−128 … +127
Unsigned word	0 … 65 535
Signed word	−32 768 … +32 767

Rule of thumb. The largest **unsigned** value in **n** bits is $\mathbf{2^n - 1}$ (8 bits ⟶ 255). A **signed** value in **n** bits ranges from -2^{n-1} to $2^{n-1} - 1$ (8 bits ⟶ −128 … +127).

Part IV · Digital Logic

13 · Digital Logic & Boolean Algebra [S31, 32, 33]

- CPUs are built from **digital logic gates** such as NAND, OR, XOR. Logic is the fundamental language of computing.
- **Boolean algebra** is the model used to design digital circuits, and the model in which CPUs operate. Basic assembly instructions perform Boolean operations.
- Boolean expressions are built from **NOT**, **AND**, **OR**.

X	Y	NOT X (¬X)	X AND Y (X ∧ Y)	X OR Y (X ∨ Y)
F	F	T	F	F
F	T	T	F	T
T	F	F	F	T
T	T	F	T	T

14 · Operator Precedence [S34]

1. Parentheses → 2. NOT → 3. AND → 4. OR

Example: `¬X ∨ Y ∧ Z` is evaluated as `(¬X) ∨ (Y ∧ Z)`.

Key Points to Remember

Topic	Remember
Levels	L4 HLL · L3 Assembly · L2 ISA · L1 Digital logic
Compiler	HLL → assembly / machine code
Assembler	Assembly → machine (object) code. MASM = Microsoft Macro Assembler
Linker	Object files + libraries → executable
Loader	Executable → memory
Assembly ↔ machine	One-to-one, processor-specific
HLL ↔ machine	One statement → one **or more** instructions
Machine language	Binary only; opcode + 0 or more operands
1 hex digit	4 bits — 2 hex digits = 1 byte
Sizes	Byte 8 · Word 16 · Dword 32 · Qword 64 · DQword 128 bits
Precedence	( ) → NOT → AND → OR

— End of Lecture 1 —